Lab01- Python 101 and Shift by N

Intro to Python

Python is an interpreted language. It doesn't get compiled before execution. This lets us create code quickly. It resembles an object-oriented language with its class definitions and encapsulation. It includes automatic garbage collection and has high-level, dynamic data types. You can use it as a scripting language or as a programming language. It uses new lines instead of semicolons to complete a command. It also uses indentation instead of curly braces to define a block of code such as loops or functions.

Because it is flexible (and easier than C), you probably will find yourself using it in the workforce. We have seen it being used increasingly in cybersecurity careers as one of many tools in your bag of tricks.

The current version of Python is 3, and for the most part that is what we expect you will be using in class. To check your version use `python3 --version`. We have installed this version on the Ubuntu 24.04 box deployed to you in iselab.ece.iastate.edu, but you are also welcome to use your own laptop or desktop in completing the programming assignments. We will not support getting Python running on your own computer nor will we provide the skeleton code outside of ISElab. You will have to export it yourself. Some of the later labs will require you to use ISElab because of special servers and their software that we will deploy to you.

In Python, you use a text editor (vim, nano, gui text editor) to create .py files and then run those files using the python interpreter. At the command line, the syntax to run what you have written is `python3 myprogram.py`.

Variables

In many programming languages such as Java or C, variable types are statically declared. You may be familiar with the following syntax to create variables:

```
// Create variables
int myNum = 15;           // Integer (whole number)
float myFloatNum = 5.99; // Floating point number
char myLetter = 'D';      // Character

// Print variables
printf("%d\n", myNum);
printf("%f\n", myFloatNum);
printf("%c\n", myLetter);
```

Unlike Java and C, Python does not require a static type. Python will automatically determine what type of variable you are working with once you assign a value to it. They also can change type after they have been set. You still use # for a comment. This can be seen below:

```
# An integer
var1 = 15

# A String
var2 = "I love cats and dogs!"

# A character
var3 = 'a'

# A boolean
var4 = True

# A list
var5 = ["apples",7,False,55,"Purple"]
```

Casting and Typing

If you want/need to specify the type of variable you cast it. To determine what type of variable you are working with you can use the type() function.

```
x = str(3)    # x will be '3'
y = int(3)    # y will be 3
z = float(3)  # z will be 3.0


print(type(x))
print(type(y))
print((type(z))
```

In future labs we will be using data types of lists, tuples, dictionaries, bytes, and bytearray. You may want to use this reference to the different types, especially because we will have to be converting between them.

Lists are similar to arrays, except each element can be a different data type. Remember, in C arrays they are all the same data type.

Tuples are like lists (arrays), but they are immutable. In other words, they can't be changed.

Set in python is similar to a set in math. It is a collection of data that has unique elements. Like a math set, you can use join operations on a set in python.

A dictionary is an ordered collection of values. It has a key:value pair relationship. The key allows it to be optimized in a search. It generally is used in JSON.

You can read more about the use of lists, tuples, sets, and dictionaries here.

```
# list
x = list(("apple", "banana", "cherry"))

# tuple
x = tuple(("apple", "banana", "cherry"))

# dictionary
x = dict(name="John", age=36)

# set
x = set(("apple", "banana", "cherry"))
```

Loops

Loops are one of the most common features used in programming languages. As you know, a for loop allows us to move over a set of values (in python a list, tuple, dictionary, set, or string). To make a For Loop in python, we use the syntax examples below. We can customize where the loop starts, and how much we increment the variable `i` each time.

```python
#Loops in increments of 1 from 0 through 9
for i in range(10):
    print(i)


#Loops by increments of 1 from 2 through 9
for i in range(2, 10, 1):
    print(i)


#Loops by increments of 3 from 0 through 9
for i in range(0, 10, 3):
    print(i)


#Loops through a list where is starts at 0 and
#ends with the value of the length of the list
list = ["apples", 7, False, 55, "Purple"]
for i in range(len(list)):
    print(list[i])
```

Sometimes in a loop you want to search for a value or a string and then exit the loop once the value or string is found. You can use the break statement for this.

```python
#One prints all the values up to the break
#The other only prints when the value is found

# In this case the values up to and including 46 will print
# I wrote this list as an ordered set, but if it was unordered
# the first time it finds 46 no matter the order it would break
def printUpTo(numList):
    for i in range(len(numList)):
        print("The value at index [",i,"] is ",numList[i],"\n")
        if numList[i] == 46:
            break

#In this case nothing prints until 46 is found.
def printOnly(numList):
    for i in range(len(numList)):
        if numList[i] == 46:
            print("The value ",numList[i]," is found at index [",i,"]\n")
            break


#Main
def main():

    numList = [40,41,42,43,44,45,46,47,48,49]
    printUpTo(numList)
    print("\n")
    printOnly(numList)

if __name__ == "__main__":
    main()
```

```
cpre331@cpre331:~/homework/lab01$ python3 demo_4.py
The value at index [ 0 ] is  40

The value at index [ 1 ] is  41

The value at index [ 2 ] is  42

The value at index [ 3 ] is  43

The value at index [ 4 ] is  44

The value at index [ 5 ] is  45

The value at index [ 6 ] is  46



The value  46  is found at index [ 6 ]
```

Functions

Functions, again, as you know, are blocks of code that are called from the main program. When done properly it makes it easier to read and write a program in Python. *When code will be used multiple times in a program it should usually be made into a function and called when needed. Without functions, it is much harder to troubleshoot and understand code.* Below are a few examples of how to write a function.

```
#Function One
def motivation():
    print("You're doing great!")


#Function Two
def alteration(input):
    myList = []
    for i in range(len(input)):
        myList.append(input[(i+1) % len(input)])
    print(myList)


#Calls the first function
motivation()


#Calls the second function
var1 = "I Like Dogs!"
alteration(var1)
```

```
cpre331@cpre331:~/homework/lab01$ python3 demo_2.py
You're doing great!
[' ', 'l', 'i', 'k', 'e', ' ', 'd', 'o', 'g', 's', '!', 'I']
```

Notice that the output for the alteration function is a list. What if you wanted a sentence structure (something more human readable)? You would need to create a string from the list. There are at least two ways to do this as shown below.

```python
def motivation():
    print("You're doing great!")

def alteration(input):
    myList = []
    for i in range(len(input)):
        myList.append(input[(i+1) % len(input)])
    return myList

def listToString(myList):
    myString = ""
    for ele in myList:
        myString +=ele
    return myString

def newListToString(myList):
    newMyString =''.join(map(str,myList))
    return(newMyString)



#Main
def main():
    motivation()

    var1 = "I like dogs!"
    outputList = alteration(var1)
    print(outputList)
    outputString = listToString(alteration(var1))
    print(outputString)
    outputNewString = newListToString(alteration(var1))
    print(outputNewString)
```

```
cpre331@cpre331:~/homework/lab01$ python3 demo_3.py
You're doing great!
[' ', 'l', 'i', 'k', 'e', ' ', 'd', 'o', 'g', 's', '!', 'I']
 like dogs!I
 like dogs!I
```

This lab has three parts. The first two are designed to help you practice your python skills. The third will use the skills to conduct cryptanalysis on a Shift by N cipher. Each exercise should be written in its own, separate python file which will be uploaded to Canvas to be graded. They should be named as Lab01_partX.py as described in the lab template at the end of this document.

Remember, do not copy your code directly from the Internet, from an AI (chatGPT for example), or from a friend. You may discuss the assignment with the TAs or your friends. **You may ask chatGPT to explain ideas to you, however, please don't copy the programs and turn them in as your own work.**

Part 01

1. Login to iselab and power on your machine "cpre3310_[netid]_Ubuntu_24.04_desktop"

2. Run the noproxy.sh script using sudo ./noproxy.sh XX.XX.XX where XX.XX.XX is the first 3 octets of your IP address range from the student network documentation.

3. Make a homework directory. Change into your homework directory. Scp the skeleton code for all three parts. It should create a new lab01 directory with the skeleton code inside. The credentials are repo/repo.

```
cpre3310@cpre3310:~/homework$ scp -r repo@repo.homework.3310.com:/home/repo/lab01/ .
```

4. You will have to use chmod to change the permissions so you can edit these files and move into the directories.

```
find /homework/lab01 -type d -exec chmod 740 {} +
find /homework/lab01 -type f -exec chmod 640 {} +
```

5. Use the `part01_skel.py` code.
    a. The program will prompt for user input. The input should be a sentence without spaces.
    b. The program passes the sentence to a modify() function. The modify() function converts characters to uppercase and returns a modified string that shifts the key *three to the left* which results in a mathematical right shift in the resulting character (the key must be to the **left** as demonstrated in class to earn credit).

Plaintext	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z
Key	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C

6. HINT: You may find the functions ord() which turns a character into ASCII and chr() which turns ASCII into a character useful in this program. Also, remember that there are 26 letters in the English alphabet and that the uppercase ASCII value of A is 65.

7. **What's the name of a Shift by N cipher with a key of 3?**

8. An example of the output can be seen below

```
cpre3310@cpre3310:~/homework/lab01/part01$ python3 part01_skel_jar.py
Write a sentence without spaces: ilikecatsanddogs
Enter the shift value (positive for mathematically shifting to the right, negative for mathematically shifti
ng to the left): 3
Encrypted sentence: LOLNHFDWVDQGGRJV
cpre3310@cpre3310:~/homework/lab01/part01$ python3 part01_skel_jar.py
Write a sentence without spaces: ILIKECATSANDDOGS
Enter the shift value (positive for mathematically shifting to the right, negative for mathematically shifti
ng to the left): 3
Encrypted sentence: LOLNHFDWVDQGGRJV
```

9. Save this file as `<netid>_lab01_part01.py` and upload it to canvas.

Part 02

1. Use the `part02_skel.py` code.
2. Given a list of lists, make a function called frequency that takes in the inputs and a character. The user will be prompted as to which character they want to search for. The function should print out (not return) the occurrences of that character in each of the lists.
3. An example of the output is shown below. While you may print it slightly differently, the values in your program should match the output below. **Include a screenshot in your lab report.**

```
cpre3310@cpre3310:~/homework/lab01/part02$ python3 part02_skel_jar.py
What character do you want to check?  A
List [0]: 9
List [1]: 0
List [2]: 11
cpre3310@cpre3310:~/homework/lab01/part02$ python3 part02_skel_jar.py
What character do you want to check?  B
List [0]: 5
List [1]: 1
List [2]: 1
cpre3310@cpre3310:~/homework/lab01/part02$ python3 part02_skel_jar.py
What character do you want to check?  C
List [0]: 0
List [1]: 7
List [2]: 2
```

4. Save this file as `<netid>_lab01_part02.py` and upload it to canvas.

Part 03

1. Use the `part03_skel.py` code and `ciphertext.txt`.
2. This program will conduct cryptanalysis on a Shift by N cipher using an exhaustive key search. **You MUST shift the key to the left which means the character is mathematically shifted right.**
3. You must **document each potential solution you arrived** at just as I demonstrated in lecture. For each attempt of the key you will include the N used in the shift and the plaintext output from the N. Your output should look like what is provided below. The example below i**s not from the ciphertext you are solving**.
4. Include this full output in your lab report. A **screenshot works**. Also include the **actual plaintext message**.

```
cpre3310@cpre3310:~/homework/lab01/part03$ python3 part03_skel_jar.py ciphertext.txt
Testing shift by 0:
NQNPJHFYXFSIITLXX

Testing shift by 1:
MPMOIGEXWERHHSKWW

Testing shift by 2:
LOLNHFDWVDQGGRJVV

Testing shift by 3:
KNKMGECVUCPFFQIUU

Testing shift by 4:
JMJLFDBUTBOEEPHTT

Testing shift by 5:
ILIKECATSANDDOGSS

Testing shift by 6:
HKHJDBZSRZMCCNFRR

Testing shift by 7:
GJGICAYRQYLBBMEQQ
```

5. Save this file as `<netid>_lab01_part03.py` and upload it to canvas.

HINT: You may find the functions ord() which turns a character into ASCII and chr() which turns ASCII into a character useful in this program. Also, remember that there are 26 letters in the English alphabet. Look back at parts 01 and 02. They provide clues on how to implement this.

NOTE: If you have a problem with starting Firefox, try following these instructions which removes the snap package for Firefox and installs it from a classic deb.

Also, if you want to remove snap completely and have it stop trying to update follow these instructions. This is supposedly what Linux Mint does to avoid using snap.

Turn-in

For your lab report, you will be working off of the Lab 01 Template that is provided at the end of this document. If you do not use the Lab 01 Template, you will be docked points. DO NOT submit your code for this class in one file. Do NOT upload as a zip file. Upload each of the python files and one PDF as needed for each lab. After you have completed your lab report, export it as a PDF file, and upload it to Canvas **before** 11:59 pm the night before your lab next week. There is a 24-hour late submission window (as noted in the syllabus) with a 4% penalty for each hour the lab submission is late.

Lab 01 Template

Answer the first three questions in a pdf file and upload it to Canvas. Questions 4 -6 remind you to upload your python code.

1.) **For Part 01, what's the name of a Shift by N cipher with a key of 3?**
(5 points)

2.) **For Part 02, include a screenshot in your lab report.**
(5 points)

3.) **Document (output copy and paste/screenshot into pdf file) each potential solution you arrived at for Part 03. For each attempt of the key you will include the N used in the shift and the plaintext output from the N. *Did you shift the key LEFT and the resulting characters RIGHT?***
(15 points)

4.) **Document in the pdf what the plaintext message is. *Did you shift the key LEFT and the resulting characters RIGHT?***
(15 points)

5.) **Submit your commented code from Part one as <netid>_lab01_part01.py to Canvas.**
(20 points)

6.) **Submit your commented code from Part two as <netid>_lab01_part02.py to Canvas.**
(20 points)

7.) **Submit your commented code from Part three as <netid>_lab01_part03.py to Canvas. *Did you shift the key LEFT and the resulting characters RIGHT?***
(20 points)